

[ESPAÇO PARA LOGO DA INSTITUIÇÃO]

UNIVERSIDADE FEDERAL DO CEARÁ

Campus de Russas

Curso de Ciência da Computação / Engenharia de Software

Disciplina: Projeto e Análise de Algoritmos

Programação Dinâmica Aplicada ao Problema da Mochila 0/1

Autor: Kumbu Gomes Kuwonza

Data: Junho de 2026

1. Introdução

A disciplina de Projeto e Análise de Algoritmos ocupa posição central na formação de estudantes de Ciência da Computação e Engenharia de Software, pois fornece as ferramentas conceituais e analíticas necessárias para projetar soluções computacionais eficientes diante de problemas com alto custo combinatório. Entre as diversas técnicas estudadas ao longo do curso, a Programação Dinâmica destaca-se por sua elegância e capacidade de transformar problemas aparentemente intratáveis em tarefas computacionalmente viáveis, por meio do reaproveitamento inteligente de resultados intermediários.

O presente trabalho tem como objetivo investigar a aplicação da técnica de Programação Dinâmica ao clássico problema da Mochila 0/1 (0/1 Knapsack Problem), um dos problemas canônicos da otimização combinatória. Busca-se, por meio deste estudo, consolidar a

compreensão teórica dos fundamentos da Programação Dinâmica e, simultaneamente, demonstrar sua aplicação prática por meio de implementações funcionais na linguagem JavaScript. O texto percorre desde a fundamentação matemática da técnica até a análise de desempenho de diferentes estratégias de codificação, passando pela exemplificação com cenários realistas de logística e gestão de recursos.

O relatório está organizado em seis seções principais. A Seção 2 estabelece a fundamentação teórica, cobrindo os conceitos gerais da Programação Dinâmica, suas duas abordagens clássicas de implementação e a formulação do problema da Mochila 0/1. A Seção 3 descreve o desenvolvimento prático realizado, incluindo a definição da recorrência, a construção passo a passo da tabela de soluções e as implementações em JavaScript com exemplos de execução. A Seção 4 apresenta a análise da complexidade computacional do algoritmo desenvolvido. A Seção 5 discute aplicações do problema da Mochila em sistemas reais. Por fim, a Seção 6 sintetiza os principais aprendizados e aponta direções para trabalhos futuros.

2. Fundamentação Teórica

2.1. Programação Dinâmica: Conceitos Gerais

A Programação Dinâmica (do inglês Dynamic Programming, abreviada como DP) é uma técnica algorítmica de otimização cuja ideia central reside em decompor um problema complexo em subproblemas menores, resolvê-los uma única vez e armazenar seus resultados para consulta futura, evitando assim a recomputação redundante. Ao contrário do que o nome sugere, o termo não se refere a uma forma de programar computadores, mas sim ao sentido matemático de "programação" como planejamento ou tomada de decisão sequencial. A técnica é particularmente eficaz quando o espaço de busca é exponencial, mas pode ser reduzido por meio do reaproveitamento de soluções intermediárias, transformando problemas aparentemente intratáveis em soluções de complexidade polinomial ou pseudo-polinomial.

A origem da Programação Dinâmica remonta à década de 1950, com os trabalhos seminais do matemático americano Richard Bellman, que cunhou o termo enquanto desenvolvia

métodos para problemas de decisão multiestágio na RAND Corporation. Bellman formulou o que hoje conhecemos como o Princípio da Otimalidade, segundo o qual "uma política ótima tem a propriedade de que, quaisquer que sejam o estado inicial e a decisão inicial, as decisões restantes devem constituir uma política ótima em relação ao estado resultante da primeira decisão". Esse princípio estabeleceu as bases teóricas para o que viria a se tornar uma das técnicas mais poderosas da ciência da computação, com aplicações que vão da bioinformática à inteligência artificial.

Duas características fundamentais determinam se um problema pode ser resolvido por Programação Dinâmica: a **sobreposição de subproblemas** e a **subestrutura ótima**. A sobreposição de subproblemas ocorre quando, ao decompor recursivamente o problema original, os mesmos subproblemas menores são gerados repetidamente: fenômeno que não ocorre, por exemplo, na ordenação por merge sort, em que cada subproblema é distinto. Já a subestrutura ótima é a propriedade que garante que a solução ótima do problema original pode ser construída a partir das soluções ótimas de seus subproblemas, permitindo a composição de respostas sem a necessidade de reavaliar decisões anteriores.

A diferença essencial entre Programação Dinâmica e a técnica de divisão-e-conquista reside justamente na natureza dos subproblemas. Na divisão-e-conquista, os subproblemas são independentes e disjuntos: pense no quicksort ou na busca binária; de modo que cada subproblema é resolvido recursivamente exatamente uma vez. Na Programação Dinâmica, os subproblemas se sobrepõem, e resolvê-los repetidamente sem memorização resultaria em explosão combinatória. A DP resolve essa ineficiência armazenando resultados em estruturas de dados (tipicamente tabelas ou matrizes) e consultando-os quando necessário, um trade-off deliberado de espaço por tempo. A técnica deve ser empregada sempre que o problema exibir as duas propriedades mencionadas e o espaço de estados for gerenciável em memória.

2.2. Memoização e Tabelaamento

A Programação Dinâmica admite duas estratégias clássicas de implementação, que diferem fundamentalmente na ordem em que os subproblemas são visitados. A primeira, conhecida como memoização (do inglês memoization), adota uma perspectiva top-down: parte-se do

problema original e recorre-se aos subproblemas menores conforme necessário, armazenando os resultados em uma estrutura de cache: tipicamente um dicionário ou vetor indexado: de modo que cada subproblema seja resolvido no máximo uma vez. A principal vantagem da memoização é sua elegância conceitual: a implementação segue naturalmente a definição recursiva do problema, preservando a clareza do raciocínio matemático original. Além disso, a memoização resolve apenas os subproblemas estritamente necessários para a solução do problema original: se determinados estados nunca são visitados, seus valores jamais são computados. A desvantagem reside no overhead das chamadas recursivas, que consomem espaço na pilha de execução e podem causar estouro de pilha para problemas com profundidade de recursão elevada.

A segunda estratégia, denominada tabelamento (do inglês *tabulation*), segue uma lógica bottom-up: os subproblemas são resolvidos em ordem crescente de tamanho, a partir dos casos base, preenchendo-se iterativamente uma tabela que contém as soluções intermediárias. A grande vantagem do tabelamento é a eliminação completa do overhead de recursão: a execução é puramente iterativa, não havendo risco de estouro de pilha, e o padrão de acesso à memória tende a ser mais favorável ao cache do processador. Como desvantagem, o tabelamento tipicamente computa todos os subproblemas do espaço de estados, mesmo aqueles que não seriam necessários para responder à consulta original. Na prática educacional e em competições de programação, o tabelamento é mais comum por sua previsibilidade e por evitar problemas com limites de recursão.

A análise de complexidade em Programação Dinâmica é parametrizada pelo número de estados distintos do problema e pelo custo de transição entre estados. Se um problema possui S estados distintos e cada transição pode ser computada em tempo T , a complexidade temporal será $O(S \cdot T)$. No caso do tabelamento, essa análise é direta: preenchemos S entradas da tabela, cada uma exigindo T operações. A complexidade espacial é $O(S)$, podendo ser reduzida quando as transições dependem apenas de um subconjunto limitado de estados anteriores. É importante notar que, para problemas cujo número de estados depende de parâmetros numéricos do domínio (como a capacidade W no problema da mochila), a complexidade resultante é pseudo-polinomial: polinomial no valor numérico da entrada, mas não no número de bits necessários para representá-la; o que distingue esses problemas dos genuinamente polinomiais.

2.3. O Problema da Mochila 0/1

O problema da Mochila 0/1 é um dos mais estudados na literatura de otimização combinatória e pertence à classe dos problemas NP-difíceis. Sua formulação clássica é notavelmente simples: dados n itens, cada qual com um peso w_i e um valor v_i , e uma mochila com capacidade máxima W , deseja-se selecionar um subconjunto de itens cujo peso total não exceda W e cujo valor total seja o maior possível. O qualificador "0/1" indica que cada item pode ser incluído no máximo uma vez: ou seja, trata-se de uma decisão binária: levar ou não levar o item. Formalmente, o problema pode ser expresso como:

Maximizar $\sum (v_i \cdot x_i)$ para i de 1 a n , sujeito a $\sum (w_i \cdot x_i) \leq W$, com $x_i \in \{0, 1\}$

A relevância deste problema no contexto acadêmico e industrial é imensa. Na disciplina de Projeto e Análise de Algoritmos, o problema da mochila 0/1 serve como exemplo paradigmático para o ensino de Programação Dinâmica porque exhibe com clareza as duas propriedades fundamentais da técnica: a subestrutura ótima (a decisão sobre levar ou não o item n reduz o problema a uma instância com $n-1$ itens e capacidade reduzida ou mantida) e a sobreposição de subproblemas (diferentes combinações de decisões levam às mesmas perguntas sobre subconjuntos de itens com capacidades residuais idênticas).

Para ilustrar o problema, considere uma transportadora que dispõe de um veículo com capacidade para 8 toneladas e precisa decidir quais fretes aceitar dentre quatro opções disponíveis, conforme a Tabela 1. O objetivo é maximizar a receita total sem exceder a capacidade do veículo.

Tabela 1: Dados do exemplo didático: fretes disponíveis

Item	Peso (ton)	Valor (R\$)
1	2	3.000
2	3	4.000
3	4	5.000
4	5	6.000

A simplicidade enunciativa do problema da Mochila esconde sua complexidade computacional. Uma abordagem ingênua por força bruta exigiria a enumeração de todos os 2^n subconjuntos possíveis de itens, resultando em complexidade exponencial $O(2^n)$. Para $n = 50$, isso representaria aproximadamente 10^{15} combinações, tornando a abordagem inviável mesmo nos computadores mais velozes disponíveis atualmente.

3. Desenvolvimento Prático

3.1. Estrutura da Solução via DP

A aplicação da Programação Dinâmica ao problema da Mochila 0/1 fundamenta-se na observação de que o problema exibe tanto a subestrutura ótima quanto a sobreposição de subproblemas. A subestrutura ótima manifesta-se da seguinte forma: ao considerar o i -ésimo item, a decisão ótima sobre incluí-lo ou não depende exclusivamente da melhor solução para os $i-1$ itens anteriores com a capacidade remanescente apropriada.

A sobreposição de subproblemas torna-se evidente quando se observa que, durante a exploração do espaço de soluções, o mesmo par (i, w) : ou seja, a mesma combinação de "quantos itens considerar" e "quanta capacidade resta": é visitado repetidamente por diferentes caminhos de decisão. Para n itens e capacidade W , existem exatamente $(n+1) \cdot (W+1)$ estados distintos no espaço de busca.

3.2. Definição da Recorrência

Seja $dp[i][w]$ o valor máximo que pode ser obtido considerando os primeiros i itens (de 1 a i) e uma mochila com capacidade w . A relação de recorrência que governa a solução é dada por:

$$dp[i][w] = \max\{ dp[i-1][w], dp[i-1][w-w_i] + v_i \}, \text{ se } w_i \leq w$$

$$dp[i][w] = dp[i-1][w], \text{ caso contrário}$$

Os casos base da recorrência são: $dp[0][w] = 0$ para todo $w \geq 0$ (sem itens, valor zero) e $dp[i][0] = 0$ para todo $i \geq 0$ (sem capacidade, valor zero).

3.3. Construção da Tabela e Reconstrução da Solução

Tabela 2: Matriz DP para o exemplo da transportadora ($n = 4$, $W = 8$)

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (item 1)	0	0	3	3	3	3	3	3	3
2 (item 2)	0	0	3	4	4	7	7	7	7
3 (item 3)	0	0	3	4	5	7	8	9	9
4 (item 4)	0	0	3	4	5	7	8	9	10

A reconstrução da solução (backtracking) percorre a tabela DP da última célula em direção aos casos base. Partindo de $dp[4][8] = 10$, a solução ótima é o conjunto {item 2, item 4}, com peso total 8 toneladas e valor R\$ 10.000,00.

3.4. Implementação em JavaScript

A implementação foi desenvolvida em JavaScript para execução em ambiente Node.js. O código-fonte completo está disponível no arquivo `knapsack.js` e está organizado em duas funções principais de solução, uma função auxiliar de exibição, e um conjunto de exemplos de execução. Adicionalmente, foi desenvolvida uma interface web interativa no arquivo `index.html` para visualização passo a passo.

3.4.1. Versão Matricial: $O(n \cdot W)$

```
function mochila01(pesos, valores, capacidade) {  
  const n = pesos.length;  
  const dp = Array.from({ length: n + 1 }, () => Array(capacidade + 1).fill(0));  
  const keep = Array.from({ length: n + 1 }, () => Array(capacidade + 1).fill(false));  
  
  for (let i = 1; i <= n; i++) {
```

```

const pesoItem = pesos[i - 1];
const valorItem = valores[i - 1];

for (let w = 0; w <= capacidade; w++) {
  if (pesoItem <= w) {
    const semItem = dp[i - 1][w];
    const comItem = dp[i - 1][w - pesoItem] + valorItem;
    if (comItem > semItem) {
      dp[i][w] = comItem;
      keep[i][w] = true;
    } else { dp[i][w] = semItem; }
  } else { dp[i][w] = dp[i - 1][w]; }
}

// Reconstrução (backtracking)
const itensSelecionados = [];
let w = capacidade;
for (let i = n; i >= 1; i--) {
  if (keep[i][w]) {
    itensSelecionados.push(i - 1);
    w -= pesos[i - 1];
  }
}
itensSelecionados.reverse();

return { valorMaximo: dp[n][capacidade], itensSelecionados };
}

```

3.4.2. Versão Otimizada: $O(W)$ de espaço

```

function mochila01Otimizada(pesos, valores, capacidade) {
  const n = pesos.length;
  const dp = Array(capacidade + 1).fill(0);
  const keep = Array.from({ length: n + 1 }, () => Array(capacidade + 1).fill(false));

  for (let i = 1; i <= n; i++) {
    const pesoItem = pesos[i - 1];
    const valorItem = valores[i - 1];

    for (let w = capacidade; w >= pesoItem; w--) {
      const comItem = dp[w - pesoItem] + valorItem;
      if (comItem > dp[w]) {
        dp[w] = comItem;
        keep[i][w] = true;
      }
    }
  }

  // Reconstrução (igual à versão matricial)
  const itensSelecionados = [];
  let w = capacidade;
  for (let i = n; i >= 1; i--) {
    if (keep[i][w]) {
      itensSelecionados.push(i - 1);
      w -= pesos[i - 1];
    }
  }
  itensSelecionados.reverse();

  return { valorMaximo: dp[capacidade], itensSelecionados };
}

```


O aspecto crucial da implementação otimizada é que o laço interno percorre as capacidades em ordem decrescente (de W até pesoItem). Se o laço fosse executado em ordem crescente, um mesmo item poderia ser incluído múltiplas vezes, convertendo o problema em uma Mochila Ilimitada.

3.4.3. Exemplos de Execução

Tabela 3: Resumo dos exemplos de execução

Cenário	n	W	Valor Ótimo	Itens	Peso Utilizado	Matricial	Otimizada
1. Caso Didático	4	8	10	[2, 4]	8/8 (100%)	0,79 ms	0,29 ms
2. Seleção de Cargas	7	50	260	[1, 6]	50/50 (100%)	0,16 ms	0,14 ms
3. Gestão de Inventário	8	30	112	[1, 2, 4, 8]	30/30 (100%)	0,30 ms	0,23 ms
4. Caso com 12 Itens	12	40	515	[1, 2, 3, 6, 8, 10, 11]	40/40 (100%)	0,07 ms	0,05 ms

Tabela 4: Comparativo de desempenho: 100 itens, $W = 500$

Métrica	Versão Matricial	Versão Otimizada
Tempo de execução	22,90 ms	6,86 ms
Valor máximo encontrado	2526	2526
Itens selecionados	35 de 100	35 de 100
Espaço para valores	$O(n \cdot W) = \sim 50.000$	$O(W) = 501$

3.4.4. Evidências de Funcionamento

Abaixo são apresentados os resultados reais da execução do código knapsack.js no ambiente Node.js, demonstrando o correto funcionamento das implementações:

```
MOCHILA 0/1 - Programação Dinâmica (Bottom-Up) ||
```

Execução do Exemplo 1:

```
=====
EXEMPLO 1 - Caso básico com 4 itens (didático)
=====
```

```
Capacidade da mochila: 8
```

```
-- Versão Matricial (O(n·W) espaço) --
```

```
Tempo (matricial): 0.793ms
```

```
Valor máximo obtido: 10
```

```
Itens selecionados: [2, 4]
```

```
Peso utilizado: 8 / 8 (100.0%)
```

```
-- Versão Otimizada (O(W) espaço para valores) --
```

```
Tempo (otimizada): 0.293ms
```

```
Valor máximo obtido: 10
```

```
Itens selecionados: [2, 4]
```

Execução do Teste Comparativo (100 itens):

```
=====
COMPARATIVO: Matricial × Otimizada
=====
```

```
Caso de teste: 100 itens, capacidade 500
```

```
Versão Matricial (O(n·W) espaço):
```

```
matricial: 22.903ms
```

```
Valor máximo: 2526
```

```
Itens selecionados: 35 de 100
```

```
Versão Otimizada (O(W) espaço para dp):
```

```
otimizada: 6.862ms
```

```
Valor máximo: 2526
```

```
Itens selecionados: 35 de 100
```

```
✓ Ambos os algoritmos retornaram o mesmo valor máximo? SIM
```

4. Análise do Algoritmo

4.1. Complexidade de Tempo

A versão matricial da Programação Dinâmica para o problema da Mochila 0/1 preenche uma tabela de dimensões $(n+1) \times (W+1)$. Para cada uma das $n \cdot W$ células efetivamente computadas, realiza-se uma quantidade constante de operações: uma comparação e, no

máximo, uma adição. Em termos assintóticos, a complexidade de tempo da solução é $\Theta(n \cdot W)$.

A versão otimizada mantém a mesma complexidade de tempo $O(n \cdot W)$, pois ainda é necessário processar cada item e cada capacidade. No entanto, o laço interno percorre apenas de W até o peso do item, o que na prática reduz ligeiramente o número de iterações. O ganho real de desempenho observado ($3,3\times$) deve-se principalmente à melhor localidade de referência e menor alocação de memória.

4.2. Complexidade de Espaço

A versão matricial armazena uma tabela de dimensões $(n+1) \times (W+1)$ para os valores DP, mais uma matriz keep para reconstrução, resultando em $O(n \cdot W)$ de memória. Para $n = 100$ e $W = 500$, são aproximadamente 50.000 células.

A versão otimizada reduz o armazenamento dos valores para um vetor unidimensional de tamanho $W+1$, alcançando $O(W)$ de espaço para os valores DP. Contudo, a matriz keep para reconstrução ainda requer $O(n \cdot W)$.

4.3. Análise Pseudo-Polinomial

A complexidade $O(n \cdot W)$ é classificada como pseudo-polinomial, e não polinomial de fato. A distinção é sutil mas importante: a análise tradicional de complexidade considera o tamanho da entrada em termos de bits necessários para representá-la, não o valor numérico dos parâmetros. Enquanto n contribui com $\log_2 n$ bits, W contribui com $\log_2 W$ bits para a representação da entrada.

4.4. Comparação com Abordagens Alternativas

Tabela 5: Comparação entre abordagens para a Mochila 0/1

Abordagem	Complexidade	Otimidade	Observação
Força Bruta	$O(2^n)$	Garantida	Inviável para $n > 40$
Guloso (valor/peso)	$O(n \log n)$	Não garantida	Rápido, mas existem contraexemplos

Branch-and-Bound	$O(2^n)$ pior caso	Garantida	Bom desempenho médio
Programação Dinâmica	$O(n \cdot W)$	Garantida	Pseudo-polinomial; limitado por W

4.5. Limitações da Implementação

- Capacidade inteira: O algoritmo requer que pesos e capacidade sejam números inteiros.
- Memória para reconstrução: A matriz keep ainda consome $O(n \cdot W)$ na versão otimizada.
- JavaScript e números grandes: O formato IEEE 754 limita a precisão para inteiros acima de 2^{53} .

5. Aplicações em Sistemas Reais

O problema da Mochila 0/1 transcende o interesse puramente acadêmico e encontra aplicações práticas em inúmeros domínios da computação e da engenharia.

Transporte e Logística: No carregamento de veículos, em que cada carga possui peso e valor de frete associado. O Exemplo 2 da Seção 3.4.3 ilustrou esse cenário, demonstrando como a DP seleciona a combinação ótima de cargas para maximizar a receita por viagem.

Gestão de Inventário: Na decisão sobre quais produtos armazenar em espaços limitados de depósito, considerando margens de lucro e giro de estoque. Supermercados e centros de distribuição utilizam princípios derivados da mochila para disposição ótima de produtos.

Sistemas Computacionais: Na alocação de máquinas virtuais a servidores físicos, seleção de funcionalidades em sistemas embarcados e corte de materiais na indústria.

Finanças: Na seleção de carteira de ativos com orçamento limitado, em que projetos competem por recursos de capital com diferentes custos e retornos esperados.

6. Conclusão

O presente trabalho percorreu o ciclo completo que vai da teoria à prática na aplicação da Programação Dinâmica ao problema da Mochila 0/1. Do ponto de vista teórico, consolidou-

se a compreensão dos princípios que fundamentam a técnica: a subestrutura ótima e a sobreposição de subproblemas:, bem como a distinção entre as abordagens top-down (memoização) e bottom-up (tabelamento), com suas respectivas vantagens e limitações.

Do ponto de vista prático, a implementação das duas versões: matricial e otimizada: evidenciou que as considerações de eficiência espacial não são meramente acadêmicas. A versão otimizada com vetor unidimensional provou ser consistentemente mais rápida que a versão matricial (aproximadamente $3,3\times$ no teste com 100 itens), com resultados idênticos.

Como possíveis extensões deste trabalho, sugere-se a investigação de variantes como a Mochila Fracionária, a Mochila Múltipla e a Mochila Multidimensional, bem como o estudo de algoritmos de aproximação FPTAS para instâncias com capacidade W muito grande.

Referências

- [1] BELLMAN, R. Dynamic Programming. Princeton: Princeton University Press, 1957.
- [2] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- [3] DASGUPTA, S.; PAPADIMITRIOU, C.; VAZIRANI, U. Algoritmos. Porto Alegre: AMGH, 2010.
- [4] KLEINBERG, J.; TARDOS, É. Algorithm Design. Boston: Pearson, 2006.
- [5] MARTELLO, S.; TOTH, P. Knapsack Problems: Algorithms and Computer Implementations. Chichester: John Wiley & Sons, 1990.
- [6] ZIVIANI, N. Projeto de Algoritmos com Implementações em Java e C++. São Paulo: Cengage Learning, 2010.